GitHub MCP Server — Study Guide

Everything built in steps 1–5: concepts, code walkthrough, commands and interview answers

Contents

How to use this guide

Part 1. Your project in one minute

- The 30-second pitch
- The 2-minute walkthrough

Part 2. Core concepts

- What MCP is and why it exists
- The three roles
- The three server primitives
- How a session works
- Transports
- Glossary

Part 3. Architecture

- Project layout

Part 3b. The toolbox: every tool and library, and why

- Runtime libraries (what the server needs to run)
- Development tools (only needed while building)
- Services and credentials

Part 4. Code walkthrough, file by file

- 4.1 `pyproject.toml` : project definition
- 4.2 `config.py` : settings
- 4.3 `github_client.py` : the HTTP layer
- 4.4 `models.py` : output shapes
- 4.5 `server.py` : the MCP layer
- 4.6 Tests
- 4.7 Why test this way
- 4.8 Manual checks we did, and why

Part 5. Commands you used

Part 6. Interview questions and model answers

- MCP fundamentals
- Tool design
- Context and output
- Engineering
- Security
- Testing
- Tools and choices
- Pagination and lists (step 4)
- Retries and rate limits (step 5)

Part 7. Practise checklist

How to use this guide

This guide covers the whole project so far: what it does, the concepts behind it, **every file of code explained line by line**, every command you ran, and interview questions with model answers. It is updated after each step.

Suggested study order:

1. Read **Part 1** and practise the pitch out loud.
2. Read **Part 2** (concepts) until you can explain each term without notes.
3. Work through **Part 4** (code walkthrough) with the real files open in VS Code.
4. Test yourself with **Part 6** (interview questions). Cover the answer, say yours, then compare.

Progress: steps 1–5 of 8 complete.

Step	What was built	Status
1	Project scaffold, config, `server_status` tool, first test	Done, verified on your laptop
2	GitHub client, `search_repositories`, `get_repository`, error handling	Done, verified live, committed
3	`list_directory`, `get_file_contents` (reading code)	Done, committed
4	`list_issues`, `list_pull_requests`, safe pagination	Done, 21 tests pass on your laptop, committed
5	Retries with backoff, rate-limit handling, `get_rate_limit`	Built, 30 tests pass
6–8	Resources & prompts, CI, Docker	Coming next

Part 1. Your project in one minute

The 30-second pitch

> *"I built an MCP server in Python that gives AI assistants like Claude safe, structured access to GitHub. It can search repositories, fetch details, browse folders, read source files, and list issues and pull requests. I used the official MCP Python SDK, an async httpx client for the GitHub REST API, and Pydantic models to keep outputs small and typed. I focused on tool design: clear names and descriptions, input validation, error messages the model can act on, and guards so large files or long lists never flood the model's context. It retries temporary failures with exponential backoff and handles GitHub's rate limits without hanging. It's covered by tests that mock GitHub, so they run offline in about a second."*

The 2-minute walkthrough

When an interviewer says *"walk me through your project"*:

1. **What MCP is.** An open protocol that standardises how AI applications connect to tools and data. You wrap a service once as an MCP server, and any MCP host can use it.
2. **Architecture.** The host (Claude Desktop or Claude Code) launches my server as a subprocess and talks JSON-RPC over stdin/stdout. My server has three layers: the MCP layer (`server.py`) defines the tools, the API layer (`github_client.py`) does HTTP, and the models layer (`models.py`) shapes what the model sees.
3. **Tools.** `search_repositories` , `get_repository` , `list_directory` , `get_file_contents` , `list_issues` , `list_pull_requests` , `get_rate_limit` , and `server_status` .
4. **Engineering decisions.** One shared async HTTP client created in the server's lifespan; outputs trimmed from ~100 fields to ~13; pagination that follows GitHub's `Link` headers with a 5-page cap; files cut off at a limit with an explicit `truncated` flag; network errors and 5xx retried with exponential backoff and jitter, rate limits waited out only when the wait is short; every failure turned into a readable tool error; a fine-grained read-only token loaded from the environment.
5. **Testing.** `pytest` with `respx` to fake GitHub (including 404s, timeouts, server errors, rate limits, binary and oversized files), calling tools through the SDK's in-process client exactly as a real host would.

Part 2. Core concepts

What MCP is and why it exists

Before MCP, every AI app needed its own custom integration for every service: N apps × M services = N×M integrations. **MCP (Model Context Protocol)** is an open standard, created by Anthropic, that turns this into N + M: each service is wrapped once as an MCP server, and every MCP-compatible app can use it. It's often described as *"USB-C for AI tools"*.

The three roles

Role	What it is	In this project
Host	The AI application the user talks to	Claude Desktop, Claude Code, the MCP Inspector
Client	A component inside the host holding one connection to one server	Built into the host; in tests, `mcp.Client`
Server	A program exposing capabilities and doing the work	Our `github-mcp` server

The three server primitives

Primitive	Who controls it	Example
Tool	The **model** decides to call it	`search_repositories`
Resource	The **application/user** attaches it as read-only context, addressed by a URI	`github://repos/{owner}/{repo}/readme` (step 6)
Prompt	The **user** picks it, like a slash command	`review_pull_request` (step 6)

This is one of the most common MCP interview questions. Memorise: *model-controlled, application-controlled, user-controlled.*

How a session works

All messages are **JSON-RPC 2.0**: requests with an `id`, `method` and `params`, and responses with a `result` or `error`.

1. The host starts our server as a subprocess.
2. `initialize`: client and server exchange protocol version and capabilities; the server sends its `instructions`.
3. `tools/list`: the server returns every tool's name, description and input/output JSON Schema.
4. The user asks something. The model reads the tool list and decides to call one.

5. `tools/call` : the host sends the tool name and arguments; our function runs and returns a result (or an error result).
6. The model reads the result and either answers or calls another tool.

Transports

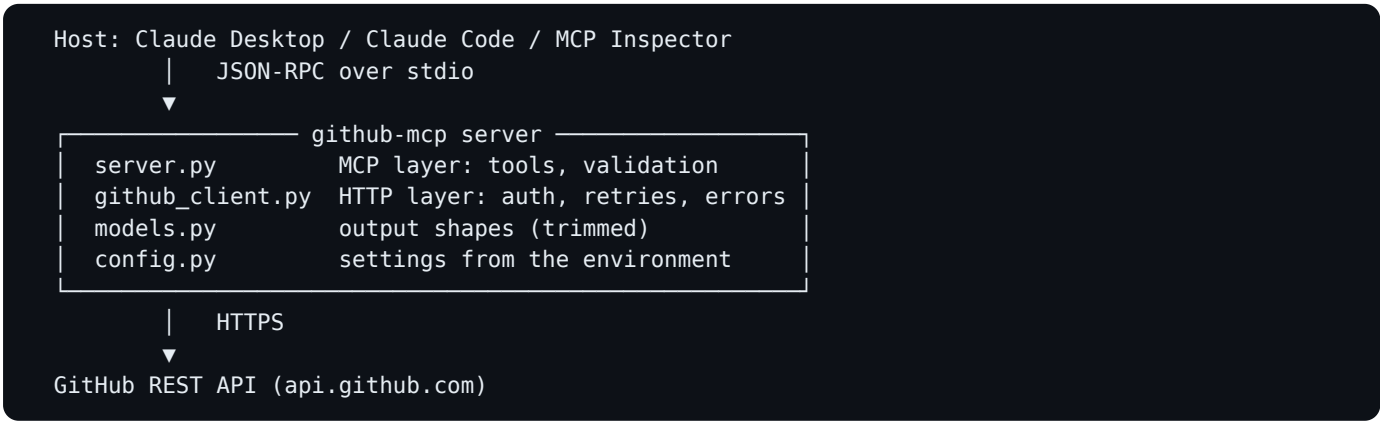
- **stdio** (what we use): the host launches the server and talks over stdin/stdout. It's local, simple, and nothing listens on the network, so the token never leaves the machine. **Consequence: stdout belongs to the protocol.** Any `print()` corrupts the stream, which is why we log to stderr.
- **Streamable HTTP** (step 8): the server runs as a web service for remote or multi-user setups. It needs authentication (OAuth) and hardening.

Glossary

Term	Meaning
Async / await	Lets one program wait for many slow network calls at once instead of one after another
Lifespan	Code that runs once at server startup and shutdown; we create and close the shared HTTP client there
Dependency injection	Giving a function the objects it needs (our HTTP client) instead of having it create them
Pydantic model	A Python class that validates data and can describe itself as a JSON Schema
Structured output	A tool result returned as typed JSON (`structuredContent`) matching a declared schema
Tool annotations	Hints like read-only or destructive that hosts use for safety and UX decisions
Base64	A way to encode any bytes (including binary) as plain text so they can travel inside JSON
Mocking	Replacing a real dependency (GitHub) with a fake that returns prepared answers, for testing
Pagination	Splitting a long list of results into pages; GitHub gives at most 100 items per page
`Link` header	A response header where GitHub says where the next page is (`rel="next"`)
Enum (`Literal`)	A fixed set of allowed values, e.g. `open` , `closed` , `all` ; shown to the model in the schema
Idempotent	Doing a request twice has the same effect as doing it once (true for GET); makes retries safe
Exponential backoff	Waiting longer after each failed attempt (0.5 s, then 1 s…) so a struggling server isn't flooded

Term	Meaning
Jitter	Randomness added to the wait so many clients don’t all retry at the same instant
Rate limit	GitHub’s cap on requests: 60/hour without a token, 5,000/hour with one
Fine-grained token	A GitHub token limited to specific permissions and repositories, with an expiry date

Part 3. Architecture



Each layer has one job:

Layer	File	Knows about	Doesn't know about
MCP	`server.py`	Tools, descriptions, validation, errors for the model	URLs, headers, HTTP
HTTP	`github_client.py`	Base URL, auth, timeouts, retries, rate limits, status codes	MCP
Models	`models.py`	Which fields the model sees	Where data comes from
Config	`config.py`	Environment variables, `.env`	Everything else

Interview line: *"I separated the protocol layer from the API layer, so each can be tested on its own and changed without touching the other."*

Project layout

```
github-mcp-server/
├── pyproject.toml          # project metadata, dependencies, tool settings
├── uv.lock                 # exact versions of every package (reproducible installs)
├── .env.example            # documents the env vars; the real .env is git-ignored
├── .gitignore              # files git must never commit (.env, .venv, caches)
├── README.md
├── src/github_mcp/
│   ├── __init__.py         # package version
│   ├── config.py           # settings from environment
│   ├── github_client.py    # async GitHub REST client
│   ├── models.py           # trimmed output models
│   └── server.py           # MCP server and tools
├── tests/
│   ├── conftest.py         # shared test fixture (fake GitHub)
│   ├── test_server.py      # step 1
│   ├── test_repositories.py# step 2
│   ├── test_contents.py    # step 3
│   ├── test_issues_and_prs.py # step 4
│   └── test_errors_and_retries.py # step 5
└── docs/                   # roadmap, step explainers, this guide
```

Why a `src/` layout? Tests import the *installed* package rather than files that happen to sit in the current folder. This catches packaging mistakes early, and it's the layout the Python Packaging Authority recommends.

Part 3b. The toolbox: every tool and library, and why

Interviewers often ask *"why did you choose X?"* A good answer names the job it does, why it fits, and an alternative you considered.

Runtime libraries (what the server needs to run)

Tool	What it does here	Why this one	Alternatives
Python 3.11+	The language	The official MCP SDK, Pydantic and most AI tooling are Python-first; 3.11+ adds faster async and modern typing (`str | None`)	TypeScript (the other official MCP SDK)
MCP Python SDK v2 (`mcp`)	Implements the protocol: JSON-RPC, handshake, schemas, transports	Official SDK, kept in step with the spec; `MCPServer` turns plain functions into tools from their type hints and docstrings	Writing JSON-RPC by hand (error-prone); third-party frameworks; the low-level `Server` API in the same SDK (more control, more code)
httpx	Makes HTTP requests to GitHub	Supports **async**, connection pooling, timeouts and base URLs; `respx` can mock it; API similar to `requests`	`requests` (no async), `aiohttp` (async but a different API, harder to mock), GitHub SDKs like PyGithub (sync, hide the HTTP details we want to control)
Pydantic v2	Validates settings, defines output models, generates JSON Schemas	The MCP SDK is built on it, so schemas and validation come for free; fast (Rust core)	dataclasses (no validation), marshmallow, attrs
python-dotenv	Loads `.env` into environment variables	Tiny, standard way to keep local secrets out of code	`pydantic-settings` (heavier), setting variables by hand each session

Development tools (only needed while building)

Tool	What it does here	Why this one	Alternatives
uv	Installs Python, creates `.venv`, installs and locks dependencies, runs commands	One fast tool replaces pip + venv + pip-tools; `uv.lock` makes installs identical everywhere; MCP hosts commonly launch servers with `uv run`	pip + venv, Poetry, pipenv, conda

Tool	What it does here	Why this one	Alternatives
pytest	Runs the tests	Python's de-facto test framework: plain `assert`, fixtures, clear failure output	`unittest` (more boilerplate)
pytest-asyncio	Lets pytest run `async def` tests	Our tools are async, so tests must be too	`anyio`'s pytest plugin
respx	Fakes GitHub's HTTP responses in tests	Built specifically for httpx; blocks real network calls; can simulate errors and timeouts	`pytest-httpx`, `unittest.mock` (low-level), VCR-style recorded responses
ruff	Lints and formats the code	Extremely fast; one tool replaces flake8, isort and black; keeps style consistent	flake8 + black + isort, pylint
MCP Inspector (`mcp dev`)	A browser UI that acts as an MCP host	Lets you call tools by hand and see the raw results without needing Claude	Testing only through Claude Desktop (slower, less visible)
Node.js	Runs the Inspector	The Inspector is a JavaScript app	—
git	Version control	Industry standard; one commit per step shows how the project grew	—
GitHub (website)	Hosts the code publicly	Where recruiters look; also the API we're integrating	GitLab, Bitbucket
VS Code	Editor	Free, good Python support	PyCharm

Services and credentials

Item	Why it's needed
GitHub REST API v3	The data source. REST is simple and well documented. (GitHub's GraphQL API can fetch nested data in one call; it's a possible later improvement.)
Fine-grained personal access token	Raises the rate limit from 60 to 5,000 requests/hour and allows access beyond anonymous limits. Fine-grained, read-only and expiring means a leak can do very little (**least privilege**).
`.env` file	Stores the token locally; listed in `.gitignore` so git never commits it.

Part 4. Code walkthrough, file by file

Open each file in VS Code next to this guide.

4.1 `pyproject.toml` : project definition

```toml
[project]
name = "github-mcp-server"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = [
    "mcp[cli]>=2.3,<3",
    "httpx>=0.28",
    "pydantic>=2.7",
    "python-dotenv>=1.0",
]

[project.scripts]
github-mcp = "github_mcp.server:main"

[dependency-groups]
dev = ["pytest>=8", "pytest-asyncio>=0.24", "respx>=0.21", "ruff>=0.6"]

[tool.pytest.ini_options]
asyncio_mode = "auto"
pythonpath = ["src"]
```

- **`pyproject.toml`** is the modern standard file describing a Python project (it replaced `setup.py`).
- **`dependencies`** are what the server needs at runtime:
  - `mcp[cli]` : the official MCP Python SDK. `[cli]` adds the `mcp` command (for `mcp dev`). `>=2.3,<3` allows bug fixes but blocks a future breaking v3.
  - `httpx` : an HTTP client with async support.
  - `pydantic` : data validation and JSON Schema generation.
  - `python-dotenv` : reads the `.env` file.
- **`[project.scripts]`** creates a command called `github-mcp` that runs `main()` in `server.py`. Hosts can launch the server by that name.
- **`[dependency-groups] dev`** holds tools needed only while developing: `pytest` (tests), `pytest-asyncio` (async tests), `respx` (fake HTTP), `ruff` (linter/formatter). Users of the server don't install these.
- **`asyncio_mode = "auto"`** lets any `async def test_...` run without a decorator. **`pythonpath = ["src"]`** lets tests import `github_mcp`.

4.2 `config.py` : settings

```python
import os

from dotenv import load_dotenv
from pydantic import BaseModel, Field, SecretStr


class Settings(BaseModel):
    github_token: SecretStr | None = None
    github_api_url: str = "https://api.github.com"
    request_timeout_s: float = Field(default=15.0, gt=0)

    @classmethod
    def from_env(cls) -> "Settings":
        # Real environment variables win over .env (override=False).
        load_dotenv(override=False)
        return cls(
            github_token=os.environ.get("GITHUB_TOKEN") or None,
            github_api_url=os.environ.get("GITHUB_API_URL", "https://api.github.com"),
            request_timeout_s=float(os.environ.get("GITHUB_TIMEOUT_S", "15")),
        )
```

- **`class Settings(BaseModel)`** : a Pydantic model, so every value is type-checked when the server starts. Bad config fails immediately and clearly instead of halfway through a request.
- **`SecretStr`** : wraps the token so that printing or logging it shows `**********`. The real value is only read with `.get_secret_value()`, in exactly one place (the HTTP client).
- **`| None = None`** : the token is optional. Without one, GitHub still works but allows only 60 requests/hour instead of 5,000.
- **`Field(default=15.0, gt=0)`** : the timeout must be greater than 0. Pydantic enforces this.
- **`@classmethod from_env`** : a constructor that builds `Settings` from the environment. Keeping this in one place means the rest of the code never touches `os.environ`.
- **`load_dotenv(override=False)`** : copies values from `.env` into the environment, **but doesn't overwrite variables that are already set**. In production, Claude Desktop passes `GITHUB_TOKEN` in its config, and that wins over any local file.
- **`os.environ.get("GITHUB_TOKEN") or None`** : turns an empty string into `None`, so an empty `GITHUB_TOKEN=` line counts as "no token".

Why environment variables? That's how MCP hosts pass secrets to servers (`env` in the host config). Never hard-code a token and never commit `.env`.

4.3 `github_client.py` : the HTTP layer

The error type

```python
class GitHubAPIError(Exception):
    """A GitHub API call failed. The message is written to be read by the model."""

    def __init__(self, status_code: int, message: str):
        super().__init__(message)
        self.status_code = status_code
```

- A **custom exception** means callers only need to catch one type for every GitHub failure (network, timeout, 404, 401…).
- It keeps the `status_code` so code can react differently later (step 5 uses it to decide what to retry).
- The message is written **for the model to read**, because the model decides what to do next.

Creating the client

```
class GitHubClient:
    def __init__(self, settings: Settings):
        headers = {
            "Accept": "application/vnd.github+json",
            "X-GitHub-Api-Version": "2022-11-28",
            "User-Agent": f"github-mcp-server/{__version__}",
        }
        if settings.github_token is not None:
            headers["Authorization"] = f"Bearer {settings.github_token.get_secret_value()}"

        self._http = httpx.AsyncClient(
            base_url=settings.github_api_url,
            headers=headers,
            timeout=settings.request_timeout_s,
        )
        self._authenticated = settings.github_token is not None  # step 5: for error hints
```

- `Accept: application/vnd.github+json` : GitHub's recommended media type for its REST API.
- `X-GitHub-Api-Version: 2022-11-28` : pins the API version, so a future GitHub change can't silently alter responses.
- `User-Agent` : GitHub rejects requests without one. Including the version helps GitHub (and you) identify traffic.
- `Authorization: Bearer ...` : added only when a token exists. This is the one place the secret is unwrapped.
- `httpx.AsyncClient(...)` : created **once** and reused for every call:
  - It keeps a **connection pool**: the TCP + TLS handshake (often 100ms+) happens once, not per request.
  - `base_url` means calls only pass the path (`/repos/...`).
  - `timeout` makes sure a hung network call can't hang the model forever.
- `self._http` : the leading underscore marks it private. Only this class touches HTTP.

Making a request

```
async def get(self, path: str, params: dict[str, Any] | None = None) -> Any:
    response = await self._get_response(path, params)
    return response.json()


async def aclose(self) -> None:
    await self._http.aclose()
```

- `get()` is for single objects (one repository, one file). It returns parsed JSON.
- `_get_response()` does the actual request, the retries and all the error handling, and returns the full response (so pagination can read its headers). Both `get()` and `get_paginated()` use it, so **error**

handling lives in exactly one place. It is explained in full in *Retries and rate limits (step 5)* below.

- `async def` / `await` : while waiting for GitHub, Python can serve other requests. Network calls are **I/O-bound** (mostly waiting), which is exactly where async helps.
- `params=params` : httpx turns a dict like `{"q": "mcp", "per_page": 5}` into `?q=mcp&per_page=5` , URL-encoding it safely.
- `aclose()` : closes the pooled connections. The server calls it at shutdown.

Pagination (step 4)

GitHub never returns a long list in one go. It returns **pages** (at most 100 items each) and puts the address of the next page in a `Link` **header**:

```
Link: <https://api.github.com/repos/o/r/issues?page=2&per_page=20>; rel="next"
```

```python
# Hard stop on how many pages one tool call may fetch, whatever the model asks for.
MAX_PAGES = 5

    async def get_paginated(
        self,
        path: str,
        params: dict[str, Any] | None = None,
        *,
        max_items: int,
        keep: Callable[[dict[str, Any]], bool] = lambda item: True,
    ) -> tuple[list[dict[str, Any]], bool]:
        url: str | None = path
        query = {**(params or {}), "per_page": min(100, max(max_items, 1))}
        items: list[dict[str, Any]] = []

        for _ in range(MAX_PAGES):
            response = await self._get_response(url, query)
            items.extend(item for item in response.json() if keep(item))
            url = response.links.get("next", {}).get("url")
            # Our token is sent with every request, so only follow links back to GitHub.
            if url is not None and not url.startswith(str(self._http.base_url).rstrip("/") + "/"):
                url = None
            # The "next" URL already carries every query parameter.
            query = None
            if url is None or len(items) >= max_items:
                break

        has_more = len(items) > max_items or url is not None
        return items[:max_items], has_more
```

- `MAX_PAGES = 5` : a hard ceiling. Whatever the model asks for, one tool call makes at most 5 requests. This bounds time, rate-limit use and context size.
- `*,` in the signature: everything after it must be passed by name (`max_items=20`), so calls read clearly.
- `keep` : an optional filter applied as items arrive. The default `lambda item: True` keeps everything; `list_issues` passes one that drops pull requests.
- `-> tuple[list[...], bool]` : returns two things, the items and whether more exist.
- `{**(params or {}), "per_page": ...}` : copies the caller's parameters and adds the page size. `min(100, ...)` respects GitHub's maximum; asking only for what we need avoids wasted data.
- `for _ in range(MAX_PAGES)` : a *bounded* loop. `while True` could run for a very long time on a big repository. `_` means "I don't need the counter".

- `items.extend(item for item in ... if keep(item))` : adds this page's items, filtered.
- `response.links` : httpx parses the `Link` header into a dict for us. `.get("next", {}).get("url")` gives the next URL, or `None` on the last page.
- **Same-host check (security)**: the client sends the token on every request. If a "next" URL ever pointed to another host, following it would leak the token. We only follow URLs starting with `https://api.github.com/` . **The trailing `/` matters**: without it, a look-alike host like `api.github.com.evil.example` would pass. A test checks exactly this.
- `query = None` : the next URL already contains all parameters; sending them again would duplicate them.
- **Stop early** once we have enough items.
- `has_more` : true if we cut items off or GitHub still had a next page. The model is told the list is partial, the same idea as `truncated` in step 3.
- `items[:max_items]` : never return more than asked.

Why follow `Link` instead of building `?page=N` ourselves? GitHub uses page numbers on most endpoints but cursors on some. Following whatever "next" link GitHub gives works with both, and if GitHub changes its scheme our code keeps working.

Retries and rate limits (step 5)

New constants at the top of the file:

```
MAX_ATTEMPTS = 3  # the first try plus two retries
BASE_DELAY_S = 0.5  # backoff ceiling doubles each time: 0.5s, then 1s
RETRYABLE_STATUS = {500, 502, 503, 504}

# A tool call should never hang for minutes. Longer rate-limit waits are reported instead.
MAX_WAIT_S = 10.0

logger = logging.getLogger("github_mcp.github_client")

# Indirection so tests can replace sleeping with a recorder and run instantly.
_sleep = asyncio.sleep
```

The error taxonomy. The question behind every decision is *"would the exact same request succeed if sent again?"*

Failure	Retry?	Why
Timeout or network error	Yes	Usually temporary
500, 502, 503, 504	Yes	GitHub's problem, often gone in a second
Rate limit with a short wait (≤ 10 s)	Wait, then yes	GitHub said when it's OK
Rate limit with a long wait	No, report it	A tool call must not hang for 30 minutes
Other 4xx (404, 401, 422, plain 403)	No	Same request, same failure; the model must change its input

Idempotency makes retrying safe: every call here is a `GET` , and doing a GET twice changes nothing. A `POST` such as `create_issue` (step 6) must not be retried blindly, because a lost response could mean the issue was

created and a retry would make a duplicate.

```python
async def _get_response(self, url: str, params: dict[str, Any] | None) -> httpx.Response:
    for attempt in range(1, MAX_ATTEMPTS + 1):
        try:
            response = await self._http.get(url, params=params)
        except httpx.TimeoutException as exc:
            error = GitHubAPIError(504, f"GitHub did not respond in time ({url}).")
            error.__cause__ = exc
        except httpx.HTTPError as exc:
            error = GitHubAPIError(502, f"Could not reach GitHub: {exc}")
            error.__cause__ = exc
        else:
            if response.is_success:
                return response

            wait = _rate_limit_wait(response)
            if wait is not None:
                if wait > MAX_WAIT_S or attempt == MAX_ATTEMPTS:
                    raise GitHubAPIError(response.status_code, self._rate_limit_message(wait))
                logger.warning("Rate limited by GitHub; waiting %.1fs before retrying", wait)
                await _sleep(wait)
                continue

            error = GitHubAPIError(response.status_code, _error_message(response))
            if response.status_code not in RETRYABLE_STATUS:
                raise error

        if attempt == MAX_ATTEMPTS:
            raise GitHubAPIError(
                error.status_code, f"{error} Gave up after {MAX_ATTEMPTS} attempts."
            ) from error.__cause__

        delay = random.uniform(0, BASE_DELAY_S * 2 ** (attempt - 1))
        logger.warning("Attempt %d failed (%s); retrying in %.2fs", attempt, error, delay)
        await _sleep(delay)

    raise AssertionError("unreachable")  # the loop always returns or raises
```

- `for attempt in range(1, MAX_ATTEMPTS + 1)` : a bounded loop, attempts 1, 2 and 3.
- `try / except / else` : `except` handles “no response at all” (network, timeout); `else` runs only when a response arrived.
- **Order of `except` matters**: `TimeoutException` is a kind of `HTTPError`, so it must be caught first to get the more specific message.
- **504 / 502**: standard HTTP gateway codes reused for “GitHub timed out” and “GitHub unreachable”.
- `error.__cause__ = exc` : attaches the original exception, the same thing `raise ... from exc` does, so tracebacks show the real cause.
- `response.is_success` : true for any 2xx status; return straight away.
- **Rate limit branch**: a short wait is slept out and the loop `continue`s to the next attempt; a long wait (or no attempts left) raises a message with the reset time instead.
- `if response.status_code not in RETRYABLE_STATUS: raise error` : other 4xx errors fail immediately. No wasted requests.
- **“Gave up after 3 attempts”**: tells the model the server already retried, so retrying again itself is pointless.
- `random.uniform(0, 0.5 * 2 ** (attempt - 1))` : **exponential backoff with full jitter**. The ceiling doubles (0.5 s, 1 s) so a struggling GitHub isn’t hammered; the random pick spreads out many clients that

failed at the same time (avoiding a “thundering herd”).

- `logger.warning`: goes to stderr, so retries show in the Inspector’s server log without breaking stdio.
- `await _sleep(...)`: `asyncio.sleep`, not `time.sleep`, so the server keeps serving other requests while waiting. Tests swap `_sleep` for a recorder.
- `raise AssertionError("unreachable")`: the loop always returns or raises; this line satisfies type checkers and would flag a logic bug loudly.

How long does GitHub want us to wait?

```python
def _rate_limit_wait(response: httpx.Response) -> float | None:
    """Seconds GitHub wants us to wait, or None if this error is not a rate limit."""
    if response.status_code not in (403, 429):
        return None
    retry_after = response.headers.get("retry-after")
    if retry_after is not None:  # secondary rate limit: GitHub says exactly how long
        try:
            return max(0.0, float(retry_after))
        except ValueError:
            return 60.0
    if response.headers.get("x-ratelimit-remaining") == "0":  # hourly budget used up
        try:
            return max(0.0, int(response.headers["x-ratelimit-reset"]) - time.time())
        except (KeyError, ValueError):
            return 60.0
    if response.status_code == 429:  # no hint given; GitHub's docs say wait a minute
        return 60.0
    return None  # a 403 without rate-limit headers is a permissions problem
```

- **Primary limit**: 60 requests/hour without a token, 5,000 with one. Every response carries `X-RateLimit-Limit`, `X-RateLimit-Remaining` and `X-RateLimit-Reset` (a Unix timestamp).
- **Secondary limits**: burst protection (too many requests too fast). GitHub answers 403 or 429, usually with `Retry-After: <seconds>`.
- **403 is ambiguous**: GitHub uses it for both “quota used up” and “no permission”. The headers tell them apart; a plain 403 returns `None` so we don’t wait for nothing.
- `max(0.0, ...)`: never a negative wait if the clock says the reset already passed.
- **Fallback of 60 seconds** when a header is malformed, following GitHub’s guidance.

```python
def _rate_limit_message(self, wait_s: float) -> str:
    minutes = max(1, round(wait_s / 60))
    message = (
        f"GitHub rate limit reached. It resets in about {minutes} minute(s). "
        "Retrying now will not help; tell the user and try again later."
    )
    if not self._authenticated:
        message += " Adding a GITHUB_TOKEN raises the limit from 60 to 5,000 requests/hour."
    return message
```

- **The error message is a prompt.** Models tend to retry failed tools, so the message says plainly what to do instead: stop and tell the user.
- `self._authenticated` is set in `__init__` from `settings.github_token is not None`; the token hint only appears when it would actually help.

Error messages

```python
def _error_message(response: httpx.Response) -> str:
    try:
        detail = response.json().get("message", "")
    except ValueError:
        detail = response.text[:200]

    hints = {
        401: "The GitHub token is missing, invalid or expired.",
        403: "Access denied. The token may not have permission for this.",
        404: "Not found. Check the owner and repository names, or the token may lack access.",
        422: "GitHub rejected the request parameters.",
        500: "GitHub had an internal error.",
        502: "GitHub is temporarily unavailable.",
        503: "GitHub is temporarily unavailable.",
        504: "GitHub timed out.",
    }
    hint = hints.get(response.status_code, "GitHub returned an error.")
    return f"{hint} (HTTP {response.status_code}: {detail})"
```

- GitHub error bodies usually look like `{"message": "Not Found"}`. If the body isn't JSON, we take the first 200 characters of text instead.
- A **plain-English hint per status code** tells the model what probably went wrong and what to try. This is what you saw in the Inspector for `nobody-xyz/nothing`.
- The raw status and GitHub's own message are kept in brackets for precision.
- Since step 5, the 403 hint only talks about permissions: real rate limits are caught earlier by `_rate_limit_wait` and get their own message.

4.4 `models.py`: output shapes

```python
class RepoSummary(BaseModel):
    full_name: str
    description: str | None
    url: str
    stars: int
    language: str | None
    updated_at: datetime

    @classmethod
    def from_api(cls, data: dict[str, Any]) -> "RepoSummary":
        return cls(
            full_name=data["full_name"],
            description=data.get("description"),
            url=data["html_url"],
            stars=data["stargazers_count"],
            language=data.get("language"),
            updated_at=data["updated_at"],
        )
```

- **Why trim?** GitHub's repository JSON has around **100 fields**, mostly URLs. Every field costs tokens in the model's context window. `RepoSummary` keeps 6.
- **Renaming** (`stargazers_count` → `stars`, `html_url` → `url`): clearer names for the model.

- **`data["..."]` vs `data.get(...)`**: square brackets for fields GitHub always sends (a missing one is a real bug and should fail loudly); `.get()` for fields that can be missing or null.
- **`updated_at: datetime`**: Pydantic parses GitHub's ISO timestamp string into a real `datetime`.
- **`from_api`**: a *factory method* that keeps the mapping from GitHub's format in one place.

```python
class Repository(RepoSummary):
    default_branch: str
    forks: int
    open_issues: int
    topics: list[str]
    license: str | None
    is_private: bool
    is_archived: bool

    @classmethod
    def from_api(cls, data: dict[str, Any]) -> "Repository":
        return cls(
            **RepoSummary.from_api(data).model_dump(),
            default_branch=data["default_branch"],
            forks=data["forks_count"],
            open_issues=data["open_issues_count"],
            topics=data.get("topics", []),
            license=(data.get("license") or {}).get("spdx_id"),
            is_private=data["private"],
            is_archived=data["archived"],
        )
```

- **Inheritance**: `Repository` *extends* `RepoSummary`, so search results stay short and the detail view adds fields on top.
- **`**RepoSummary.from_api(data).model_dump()`**: builds the summary fields, turns them into a dict, and unpacks them as arguments. No duplicated mapping code.
- **`(data.get("license") or {}).get("spdx_id")`**: GitHub sends `"license": null` for repos without one. `or {}` swaps `None` for an empty dict, so `.get("spdx_id")` safely returns `None` instead of crashing. That's why you saw `"license": "MIT"`.

```python
class RepoSearchResult(BaseModel):
    total_count: int
    items: list[RepoSummary]


class DirectoryEntry(BaseModel):
    name: str
    path: str
    type: str  # "file", "dir", "symlink" or "submodule"
    size: int


class DirectoryListing(BaseModel):
    repository: str
    path: str
    entries: list[DirectoryEntry]


class FileContents(BaseModel):
    repository: str
    path: str
    size: int
    is_binary: bool
    truncated: bool
    content: str | None
```

Step 4 added four more models:

```python
class IssueSummary(BaseModel):
    number: int
    title: str
    state: str
    author: str | None
    labels: list[str]
    comments: int
    created_at: datetime
    updated_at: datetime
    url: str
    # from_api: author=(data.get("user") or {}).get("login"),
    #           labels=[label["name"] for label in data.get("labels", [])]


class PullRequestSummary(BaseModel):
    number: int
    title: str
    state: str
    author: str | None
    draft: bool
    head: str  # from data["head"]["ref"]: the branch with the changes
    base: str  # from data["base"]["ref"]: the branch it merges into
    created_at: datetime
    updated_at: datetime
    url: str


class IssueList(BaseModel):  # PullRequestList has the same shape
    repository: str
    state: str
    returned: int
    has_more: bool
    items: list[IssueSummary]
```

- About **10 fields** each, out of the 60–80 GitHub sends.
- **`(data.get("user") or {}).get("login")`** : GitHub sends `"user": null` for deleted accounts; this avoids a crash.
- **`labels`** : GitHub sends label objects with colours and URLs; we keep just the names.
- **`head`** / **`base`** : what you need to describe a pull request (“merges `feature-x` into `main`”).
- **`returned`** and **`has_more`** : tell the model how many it got and whether there are more.
- **`total_count`** + **`items`** : tells the model “70,718 matches, here are 5”, so it knows results are partial.
- **`FileContents.truncated`** and **`is_binary`** : flags that tell the model *what it didn’t get*, which is essential to stop it reasoning from incomplete data.
- **Why Pydantic models instead of dicts?** The SDK turns the return type into an **output schema** and returns **`structuredContent`** : typed, predictable JSON. In step 1 our test proved a plain dict produced no structured output.

Step 5 added one more model, for the new `get_rate_limit` tool:

```python
class RateLimitStatus(BaseModel):
    authenticated: bool
    limit: int  # requests allowed per hour
    remaining: int
    used: int
    resets_at: datetime
    search_remaining: int  # search has its own, smaller per-minute budget

    @classmethod
    def from_api(cls, data: dict[str, Any], authenticated: bool) -> "RateLimitStatus":
        core = data["resources"]["core"]
        return cls(
            authenticated=authenticated,
            limit=core["limit"],
            remaining=core["remaining"],
            used=core["used"],
            resets_at=datetime.fromtimestamp(core["reset"], UTC),
            search_remaining=data["resources"]["search"]["remaining"],
        )
```

- `core` is the main REST budget; `search` is separate (30 requests per minute), so both are reported.
- `datetime.fromtimestamp(..., UTC)`: GitHub sends the reset as a Unix timestamp (seconds since 1970); converting it gives the model a readable date-time with an explicit time zone.

4.5 `server.py`: the MCP layer

Logging and settings

```python
# stdio transport uses stdout for protocol messages, so logs must go to stderr.
logging.basicConfig(level=logging.INFO, stream=sys.stderr)
logger = logging.getLogger("github_mcp")

settings = Settings.from_env()
```

- With stdio, **stdout carries JSON-RPC**. Logging to stdout would corrupt the protocol, so all logs go to **stderr**. The host shows stderr in its logs.
- `settings` is loaded once when the module is imported.

Lifespan: shared resources

```python
@dataclass
class AppContext:
    github: GitHubClient


@asynccontextmanager
async def lifespan(_server: MCPServer) -> AsyncIterator[AppContext]:
    """Create shared resources once at startup and close them cleanly at shutdown."""
    github = GitHubClient(settings)
    logger.info("GitHub client ready (authenticated=%s)", settings.github_token is not None)
    try:
        yield AppContext(github=github)
    finally:
        await github.aclose()
```

- `@dataclass AppContext` : a simple typed container for everything tools share. More shared objects (a cache, for example) can be added later.
- `@asynccontextmanager` : code **before** `yield` **runs at startup**; code **after it (in** `finally` **) runs at shutdown**, even if the server crashes.
- `yield AppContext(...)` : the SDK stores this object and makes it available to every tool call.
- `try / finally` : guarantees the connections are closed.
- Logging `authenticated=True/False` and **never the token itself**.

Creating the server

```
mcp = MCPServer(
    name="github",
    version=__version__,
    instructions=(
        "Tools for exploring GitHub repositories, issues and pull requests. "
        "Use search_repositories to find repositories, then get_repository for details. "
        "To read code, call list_directory to explore, then get_file_contents for one file. "
        "list_issues and list_pull_requests return the most recent first; has_more=true "
        "means there are more than you were given. "
        "If a tool reports a rate limit, call get_rate_limit and tell the user when it resets "
        "instead of retrying."
    ),
    lifespan=lifespan,
)

READ_ONLY = ToolAnnotations(read_only_hint=True, open_world_hint=True)


def _github(ctx: Context) -> GitHubClient:
    return ctx.request_context.lifespan_context.github
```

- `MCPServer` : the high-level server class in **MCP Python SDK v2**. In v1 it was called `FastMCP` . Most tutorials online still show v1, so mentioning that you used the current v2 API is a good detail.
- `instructions` : sent to the host during `initialize` . It's guidance that spans several tools: *the workflow*, search then get, list then read, and (since step 5) what to do about a rate limit.
- `ToolAnnotations(read_only_hint=True, open_world_hint=True)` : tells hosts these tools don't change anything and talk to an external system. Hosts can skip confirmation prompts for read-only tools. These are **hints, not security**.
- `_github(ctx)` : a small helper that fetches the shared client from the lifespan context. This is **dependency injection**: tools receive the client and don't create it.

Shared parameter types

```
Owner = Annotated[
    str, Field(description="Account or organisation that owns the repo, e.g. 'pallets'")
]
Repo = Annotated[str, Field(description="Repository name, e.g. 'flask'")]
Ref = Annotated[
    str | None,
    Field(description="Branch, tag or commit SHA. Defaults to the repository's default branch."),
]
```

- `Annotated[type, Field(...)]` attaches extra information to a type. The SDK copies the `description` into the tool's JSON Schema, which **the model reads when deciding how to call the tool**.
- Defined **once, reused by every tool** (DRY): consistent descriptions everywhere.
- **Examples in descriptions** (`'pallets'` , `'flask'`) noticeably improve how well models fill in arguments.

Tool 1: `server_status`

```python
class ServerStatus(BaseModel):
    version: str
    github_api_url: str
    authenticated: bool


@mcp.tool()
def server_status() -> ServerStatus:
    """Report whether the server is running and whether a GitHub token is configured."""
    return ServerStatus(
        version=__version__,
        github_api_url=settings.github_api_url,
        authenticated=settings.github_token is not None,
    )
```

- `@mcp.tool()` registers the function as a tool. The SDK reads:
  - the **function name** → tool name,
  - the **docstring** → tool description (what the model sees),
  - the **type hints** → input JSON Schema,
  - the **return type** → output JSON Schema.
- **Key insight: the docstring and parameter descriptions are the prompt.** They're how the model knows what the tool does.
- Reports `authenticated` as a boolean, **never the token**.

Tool 1b: `get_rate_limit` (step 5)

```python
@mcp.tool(annotations=READ_ONLY)
async def get_rate_limit(ctx: Context) -> RateLimitStatus:
    """Show how many GitHub API requests are left this hour and when the budget resets.
    Checking it does not use up any requests."""
    try:
        data = await _github(ctx).get("/rate_limit")
    except GitHubAPIError as exc:
        raise ToolError(str(exc)) from exc
    return RateLimitStatus.from_api(data, authenticated=settings.github_token is not None)
```

- `GET /rate_limit` is free: GitHub doesn't count it against your budget, which the docstring tells the model so it isn't afraid to call it.
- **Why a tool?** The model can check the budget before a big exploration, or explain a rate-limit error to the user with the exact reset time.
- Same pattern as every other tool: shared client from `ctx` , `GitHubAPIError` → `ToolError` , Pydantic model out.

Tool 2: `get_repository`

```python
@mcp.tool(annotations=READ_ONLY)
async def get_repository(
    owner: Owner,
    repo: Repo,
    ctx: Context,
) -> Repository:
    """Get details of one GitHub repository: description, stars, language, topics, license,
    default branch and open issue count."""
    try:
        data = await _github(ctx).get(f"/repos/{owner}/{repo}")
    except GitHubAPIError as exc:
        raise ToolError(str(exc)) from exc
    return Repository.from_api(data)
```

- `async def` : the tool awaits a network call.
- `ctx: Context` : a parameter typed `Context` is **injected by the SDK and hidden from the model**. It doesn't appear in the input schema.
- **The docstring lists what's returned**, so the model knows whether this tool answers its question.
- `raise ToolError(...)` : converts our error into a tool result with `isError: true` and a readable message. The model reads it and can recover (fix a typo, try another repo). The server doesn't crash, and no stack trace leaks.
- `Repository.from_api(data)` : trims ~100 fields down to 13.

Tool 3: `search_repositories`

```python
@mcp.tool(annotations=READ_ONLY)
async def search_repositories(
    query: Annotated[
        str,
        Field(
            description=(
                "GitHub search syntax. Examples: 'mcp server language:python', "
                "'topic:llm stars:>1000', 'user:anthropics'"
            )
        ),
    ],
    ctx: Context,
    limit: Annotated[int, Field(ge=1, le=30, description="Maximum results to return")] = 10,
) -> RepoSearchResult:
    """Search public GitHub repositories, sorted by best match. Returns a short summary of
    each match; call get_repository for full details of one."""
    try:
        data = await _github(ctx).get(
            "/search/repositories", params={"q": query, "per_page": limit}
        )
    except GitHubAPIError as exc:
        raise ToolError(str(exc)) from exc
    return RepoSearchResult(
        total_count=data["total_count"],
        items=[RepoSummary.from_api(item) for item in data["items"]],
    )
```

- **The `query` description teaches GitHub search syntax** with examples. Models write much better queries when shown the syntax.

- `Field(ge=1, le=30)` : `limit` must be between 1 and 30. The SDK rejects `limit=500` **before our code runs**. This bounds cost and context size.
- `= 10` : a sensible default, so the model can leave it out.
- **The docstring points to the next tool** (“call get_repository for full details”): search returns summaries, get returns details.
- `per_page` : GitHub’s parameter for how many results to return.
- **List comprehension** `[RepoSummary.from_api(item) for item in data["items"]]` trims every result.

Shared helper for file contents

```python
async def _get_contents(ctx: Context, owner: str, repo: str, path: str, ref: str | None):
    # quote() escapes spaces and special characters but keeps "/" between folders.
    path = path.strip("/")
    url = f"/repos/{owner}/{repo}/contents" + (f"/{quote(path)}" if path else "")
    params = {"ref": ref} if ref else None
    try:
        return await _github(ctx).get(url, params=params)
    except GitHubAPIError as exc:
        raise ToolError(str(exc)) from exc
```

- Both step 3 tools call the same GitHub endpoint: `GET /repos/{owner}/{repo}/contents/{path}` . For a **folder** it returns a JSON **list**; for a **file**, a JSON **object** with base64 content.
- `path.strip("/")` : accepts `"/src/"` , `"src/"` or `"src"` alike.
- `quote(path)` : URL-encodes spaces and special characters (`my file.md` → `my%20file.md`) but keeps `/` .
- **Empty path** → no trailing slash → the repository root.
- `ref` (branch, tag or commit) is only sent when given; otherwise GitHub uses the default branch.

Tool 4: `list_directory`

```python
@mcp.tool(annotations=READ_ONLY)
async def list_directory(
    owner: Owner,
    repo: Repo,
    ctx: Context,
    path: Annotated[str, Field(description="Folder path; empty for the repository root")] = "",
    ref: Ref = None,
) -> DirectoryListing:
    """List the files and folders at a path in a repository. Use it to explore the
    structure of a codebase before reading individual files with get_file_contents."""
    data = await _get_contents(ctx, owner, repo, path, ref)
    if not isinstance(data, list):
        raise ToolError(f"'{path}' is a file, not a folder. Use get_file_contents to read it.")

    # Folders first, then files, each alphabetically: easier for the model to scan.
    entries = sorted(data, key=lambda e: (e["type"] != "dir", e["name"].lower()))
    return DirectoryListing(
        repository=f"{owner}/{repo}",
        path=path.strip("/") or "/",
        entries=[
            DirectoryEntry(name=e["name"], path=e["path"], type=e["type"], size=e["size"])
            for e in entries
        ],
    )
```

- `isinstance(data, list)` : tells a folder (list) from a file (object).
- **Error that redirects**: if the model calls this on a file, the message names the right tool. The model fixes its own mistake on the next call.
- **Sorting with a tuple key** `(e["type"] != "dir", e["name"].lower())` :
  - Python sorts tuples element by element.
  - `e["type"] != "dir"` is `False` for folders and `True` for files, and `False < True` , so **folders come first**.
  - `.lower()` makes the alphabetical order case-insensitive.
- `path.strip("/") or "/"` : shows `/` for the root instead of an empty string.

Tool 5: `get_file_contents`

```python
@mcp.tool(annotations=READ_ONLY)
async def get_file_contents(
    owner: Owner,
    repo: Repo,
    path: Annotated[
        str, Field(description="File path from the repository root, e.g. 'src/app.py'")
    ],
    ctx: Context,
    ref: Ref = None,
    max_chars: Annotated[
        int,
        Field(ge=1_000, le=100_000, description="Cut the file off after this many characters"),
    ] = 20_000,
) -> FileContents:
    """Read the text of one file in a repository. Long files are cut off at max_chars
    (truncated=true tells you this happened). Binary files return no content."""
    data = await _get_contents(ctx, owner, repo, path, ref)
    if isinstance(data, list):
        raise ToolError(f"'{path}' is a folder. Use list_directory to see what's inside.")
    if data.get("type") != "file":
        raise ToolError(f"'{path}' is a {data.get('type')}, not a regular file.")
```

- `max_chars` **between 1,000 and 100,000, default 20,000**: the model can ask for more when needed, but never unlimited. `20_000` is just `20000` with an underscore for readability.
- **The docstring explains the flags**, so the model knows what `truncated` and binary mean.
- **Two guard clauses**: a folder → redirect to `list_directory` ; a symlink or submodule → clear error.

```
    result = FileContents(
        repository=f"{owner}/{repo}",
        path=data["path"],
        size=data["size"],
        is_binary=False,
        truncated=False,
        content=None,
    )

    # GitHub only inlines files up to 1 MB; bigger ones come back with encoding "none".
    if data.get("encoding") != "base64":
        raise ToolError(
            f"'{path}' is {data['size']:,} bytes, too large to read through the GitHub API (1 MB
limit)."
        )

    raw = base64.b64decode(data["content"])
    text = _decode_text(raw)
    if text is None:
        result.is_binary = True
        return result

    result.truncated = len(text) > max_chars
    result.content = text[:max_chars]
    return result
```

- **Start with safe defaults**, then fill in. Every return path gives a complete, valid result.
- **Files over 1 MB**: GitHub's contents API doesn't include their content (`encoding: "none"`), so we explain this instead of returning nothing. `{:,}` formats `5000000` as `5,000,000` .
- **`base64.b64decode`** : GitHub sends file bytes as base64 text inside JSON. We decode them back to raw bytes.
- **Binary files** return `is_binary: true` and no content, since base64 or garbled bytes waste tokens and are useless to the model.
- **`result.truncated = len(text) > max_chars`** : an explicit flag. **Silently cutting text would let the model think it saw the whole file.**
- **`text[:max_chars]`** : Python slicing; it's safe even when the text is shorter.

```
def _decode_text(raw: bytes) -> str | None:
    """Return the bytes as UTF-8 text, or None if they look like a binary file."""
    if b"\x00" in raw:  # text files practically never contain null bytes
        return None
    try:
        return raw.decode("utf-8")
    except UnicodeDecodeError:
        return None
```

- **Binary detection heuristic**: a null byte (`\x00`), or bytes that aren't valid UTF-8, means binary. Git uses the same idea.
- Returning **`None`** (instead of raising) keeps the caller simple: `if text is None` .

Shared types for step 4

```
State = Annotated[
    Literal["open", "closed", "all"], Field(description="Filter by state. Defaults to open.")
]
Limit = Annotated[int, Field(ge=1, le=100, description="Maximum items to return, newest first")]
```

- `Literal["open", "closed", "all"]` becomes an **enum** in the JSON Schema: the model sees exactly which values exist, and anything else (like `"merged"`) is rejected before our code runs.
- `Limit`: 1 to 100, reused by both tools.

Tool 6: `list_issues`

```
@mcp.tool(annotations=READ_ONLY)
async def list_issues(
    owner: Owner,
    repo: Repo,
    ctx: Context,
    state: State = "open",
    labels: Annotated[
        str | None, Field(description="Comma-separated label names, e.g. 'bug,help wanted'")
    ] = None,
    limit: Limit = 20,
) -> IssueList:
    """List issues in a repository, newest first. Pull requests are left out; use
    list_pull_requests for those."""
    params = {"state": state, "sort": "created", "direction": "desc"}
    if labels:
        params["labels"] = labels
    try:
        # GitHub's issues endpoint also returns pull requests (they carry a
        # "pull_request" key), so we filter them out while paging.
        items, has_more = await _github(ctx).get_paginated(
            f"/repos/{owner}/{repo}/issues",
            params,
            max_items=limit,
            keep=lambda item: "pull_request" not in item,
        )
    except GitHubAPIError as exc:
        raise ToolError(str(exc)) from exc
    return IssueList(
        repository=f"{owner}/{repo}",
        state=state,
        returned=len(items),
        has_more=has_more,
        items=[IssueSummary.from_api(item) for item in items],
    )
```

- **Defaults** (`open`, 20 items) cover the most common question: “what’s open right now?”
- `sort=created`, `direction=desc`: newest first, and the docstring says so, so the model can interpret the order.
- `labels` only sent when given. GitHub expects a comma-separated string.
- **Why drop pull requests?** In GitHub’s data model **every pull request is also an issue**, so `/issues` returns both. Pull requests carry a `pull_request` key. Mixing them in would confuse the model and use

up its limit, so `keep=` filters them out *while paging* (so we still collect enough real issues), and the docstring points to `list_pull_requests`.

Tool 7: `list_pull_requests`

```python
@mcp.tool(annotations=READ_ONLY)
async def list_pull_requests(
    owner: Owner,
    repo: Repo,
    ctx: Context,
    state: State = "open",
    limit: Limit = 20,
) -> PullRequestList:
    """List pull requests in a repository, newest first, with their branch names and
    draft status."""
    params = {"state": state, "sort": "created", "direction": "desc"}
    try:
        items, has_more = await _github(ctx).get_paginated(
            f"/repos/{owner}/{repo}/pulls", params, max_items=limit
        )
    except GitHubAPIError as exc:
        raise ToolError(str(exc)) from exc
    return PullRequestList(
        repository=f"{owner}/{repo}",
        state=state,
        returned=len(items),
        has_more=has_more,
        items=[PullRequestSummary.from_api(item) for item in items],
    )
```

- The same pattern as `list_issues` on the `/pulls` endpoint. Reusing `get_paginated` means paging, the page cap, the host check and error handling all come for free.
- The **server** `instructions` were also updated to say these tools return newest first and what `has_more` means.

Entry point

```python
def main() -> None:
    logger.info("Starting GitHub MCP server v%s", __version__)
    mcp.run()  # stdio by default


if __name__ == "__main__":
    main()
```

- `main()` is what the `github-mcp` command runs (see `[project.scripts]`).
- `mcp.run()` starts the stdio transport and serves requests until the host closes it.
- `if __name__ == "__main__"` also lets you run the file directly.

4.6 Tests

`conftest.py` : the fake GitHub

```python
import pytest
import respx

from github_mcp import github_client
from github_mcp.server import settings


@pytest.fixture
def github_api():
    """Intercept every HTTP call to the GitHub API; unmatched calls fail the test."""
    with respx.mock(base_url=settings.github_api_url, assert_all_called=False) as router:
        yield router


@pytest.fixture(autouse=True)
def sleeps(monkeypatch):
    """Replace real waiting with a list of the delays asked for, so retry tests run instantly."""
    delays: list[float] = []

    async def fake_sleep(seconds: float) -> None:
        delays.append(seconds)

    monkeypatch.setattr(github_client, "_sleep", fake_sleep)
    return delays
```

- `conftest.py` : pytest automatically shares fixtures defined here with every test file.
- **A fixture** is setup code a test asks for by naming it as a parameter (`def test_x(github_api)`).
- `respx.mock` intercepts every httpx request. Tests declare which URLs return what, and **any unexpected request fails the test**, so tests can never touch the real internet.
- **Why mock?** Tests are fast (~1 second), work offline and in CI, never hit rate limits, and can simulate failures (404, timeouts) that are hard to trigger for real.
- `sleeps` **(step 5)**: `autouse=True` applies it to **every** test without asking. `monkeypatch.setattr` swaps the client's `_sleep` for a fake that only records the delay, and undoes the swap after each test. Retry tests never really wait, and can assert exactly how long the code *would* have waited. **Never make tests actually sleep**: it makes suites slow and flaky.

`test_server.py` : the wiring

```python
from mcp import Client

from github_mcp.server import mcp


async def test_server_status_tool_is_listed_and_callable():
    async with Client(mcp) as client:
        tools = await client.list_tools()
        assert "server_status" in [t.name for t in tools.tools]

        result = await client.call_tool("server_status", {})
        assert result.is_error is False
        assert result.structured_content["version"] == "0.1.0"
```

- `Client(mcp)` connects to the server **in-process**: no subprocess, no network. It does exactly what a host does (`initialize` , `tools/list` , `tools/call`).
- This tests the **whole MCP path**: registration, schema, calling, structured output.

`test_repositories.py` : example test

```python
async def test_get_repository_not_found_is_a_readable_tool_error(github_api):
    github_api.get("/repos/nobody/nothing").respond(404, json={"message": "Not Found"})

    async with Client(mcp) as client:
        result = await client.call_tool("get_repository", {"owner": "nobody", "repo": "nothing"})

    assert result.is_error is True
    assert "Not found" in result.content[0].text
```

The **Arrange / Act / Assert** pattern:

1. **Arrange**: tell the fake GitHub to answer this URL with a 404.
2. **Act**: call the tool through the MCP client.
3. **Assert**: the result is an error result (not a crash) with a readable message.

Test	What it checks	Why it's needed
`test_get_repository_returns_trimmed_fields`	Fields are renamed (`stargazers_count` → `stars`), the license is extracted, and extras like `node_id` are dropped	Proves the context-saving trimming works; if GitHub's field names were mapped wrongly, the model would get missing or wrong data
`test_get_repository_not_found_is_a_readable_tool_error`	A 404 returns `is_error: true` with "Not found…"	The model must receive a message it can act on, not a crash; this is the most common real-world failure (typos)
`test_search_repositories_passes_query_and_limit`	The query and limit reach GitHub as `q` and `per_page`	A wrong parameter name would silently return wrong results; checking the outgoing request catches it

Test	What it checks	Why it's needed
`test_search_repositories_rejects_limit_over_30`	`limit=500` is rejected before any HTTP call	Proves schema validation guards cost and context size, even if the model asks for too much
`test_timeout_becomes_a_tool_error`	A network timeout becomes a readable tool error	Networks fail; the server must stay up and tell the model what happened. Timeouts can't be triggered reliably for real, which is why we mock them

`test_contents.py` : step 3 tests

```python
def file_response(path: str, data: bytes) -> dict:
    return {
        "type": "file",
        "path": path,
        "size": len(data),
        "encoding": "base64",
        "content": base64.b64encode(data).decode(),
    }
```

A small helper that builds a fake GitHub file response (encoding bytes to base64, just like GitHub does), so each test stays short.

Test	What it checks	Why it's needed
`test_list_directory_puts_folders_first`	Results are ordered folders first, then files, alphabetically	Locks in the ordering that makes listings easy for the model to scan
`test_list_directory_on_a_file_points_to_get_file_contents`	Calling it on a file returns an error naming `get_file_contents`	Proves the model gets told how to fix its own mistake
`test_get_file_contents_decodes_text_and_passes_ref`	base64 is decoded to the right text, `truncated` is false, and `ref=v3.0.0` is sent to GitHub	The core happy path; also proves branch/tag reading works

Test	What it checks	Why it's needed
`test_get_file_contents_truncates_long_files`	A 5,000-character file with `max_chars=1000` returns exactly 1,000 characters and `truncated: true`	The main context guard; the flag must be set or the model would think it saw the whole file
`test_get_file_contents_does_not_return_binary_data`	A PNG returns `is_binary: true` and no content	Stops useless bytes wasting tokens
`test_get_file_contents_on_a_folder_points_to_list_directory`	Calling it on a folder returns an error naming `list_directory`	Same self-correction idea, in the other direction
`test_get_file_contents_reports_files_over_1mb`	GitHub's `encoding: "none"` response gives a clear "too large" error	Without it, big files would fail confusingly; this case is rare, so a test is the only reliable way to check it

`test_issues_and_prs.py` : step 4 tests

Pages are simulated with respx's `side_effect` : a function that looks at the request's `page` parameter and returns the matching fake response, each with its own `Link` header.

Test	What it checks	Why it's needed
`test_list_issues_trims_fields_and_drops_pull_requests`	PRs removed; author and label names mapped	Proves the "every PR is an issue" quirk is handled
`test_list_issues_sends_state_labels_and_newest_first`	`state` , `labels` , `sort` , `direction` reach GitHub	Wrong parameter names would silently give wrong results

Test	What it checks	Why it's needed
`test_list_issues_follows_link_header_until_limit`	Follows `Link` across pages, **stops after 2 requests** once it has 3 items, `has_more` true	Core pagination behaviour with no wasted requests
`test_pagination_stops_at_max_pages`	A repo that always has another page gets exactly 5 requests	Proves the safety ceiling works
`test_list_issues_rejects_unknown_state`	`state="merged"` is rejected	Proves the enum validation
`test_list_pull_requests_returns_branches_and_draft`	`head`, `base`, `draft`, and `state` / `per_page` sent	The PR happy path
`test_list_pull_requests_missing_repo_is_a_tool_error`	404 becomes a readable error	Error path for the new tool
`test_pagination_never_follows_links_to_other_hosts`	A "next" link to `api.github.com.evil.example` is not followed	Security: the token must never go to another host. **Verified that this test fails if the trailing `/` is removed from the check**, so it really guards the bug

That last point is a useful habit worth mentioning in interviews: **a test you've never seen fail might not be testing anything.** Break the code on purpose once and confirm the test catches it.

`test_errors_and_retries.py` : step 5 tests

Two respx tricks: `route.side_effect = [response1, response2]` returns the responses **in order** (fail, then succeed), and `route.call_count` says how many requests were really made.

```python
async def test_server_error_is_retried_then_succeeds(github_api, sleeps):
    route = github_api.get("/repos/pallets/flask")
    route.side_effect = [httpx.Response(503), httpx.Response(200, json=FLASK)]

    result = await call_get_repository()

    assert result.is_error is False
    assert route.call_count == 2
    assert len(sleeps) == 1 and 0 <= sleeps[0] <= 0.5
```

Test	What it checks	Why it's needed
`test_server_error_is_retried_then_succeeds`	503 then 200 → success, 2 requests, one wait ≤ 0.5 s	The basic retry path
`test_backoff_doubles_between_attempts`	With `random.uniform` patched to return its maximum, waits are exactly `[0.5, 1.0]`	Proves the backoff is exponential
`test_gives_up_after_max_attempts`	Always 503 → exactly 3 requests, "Gave up after 3 attempts"	Retries are bounded
`test_timeout_is_retried`	A timeout, then 200 → success	Network failures are retried too
`test_not_found_is_not_retried`	404 → 1 request, no waiting	Don't waste requests on errors that won't change
`test_short_retry_after_is_waited_out`	429 with `Retry-After: 2` → waits exactly 2 s, then succeeds	Respects GitHub's instruction
`test_exhausted_hourly_limit_is_reported_not_waited`	403 with `Remaining: 0`, reset in 30 min → 1 request, no wait, "30 minute" in the message	A tool call never hangs for minutes
`test_forbidden_without_rate_limit_headers_is_not_retried`	Plain 403 → "Access denied", no wait	Tells permission errors apart from rate limits
`test_get_rate_limit_reports_core_budget`	Numbers and reset date mapped from `/rate_limit`	The new tool's happy path

Same habit as step 4: **removing the "don't retry other 4xx" line makes the 404 and plain-403 tests fail**, so they really guard that rule.

4.7 Why test this way

- **Every test goes through the real MCP path** (`Client` → `tools/call` → our function), not by calling functions directly. That way the tests also cover tool registration, schemas, validation and error conversion.
- **Each guard has its own test.** A guard nobody tests tends to break quietly when code changes.
- **Success and failure paths.** Most bugs in API integrations hide in the error handling, so roughly half the tests are failure cases.
- **Fast and offline.** The whole suite runs in about a second with no internet or token, so you can run it after every change, and later in CI (step 7).

4.8 Manual checks we did, and why

Automated tests use a *fake* GitHub. These manual checks proved the server works against the **real** one, as a real host would use it.

Check	Result you saw	Why we did it
`uv run pytest` on your laptop	1, then 6, then 13, then 21 passed	Proves the code runs on Windows with your Python, not just in my environment
Inspector → `server_status`	`version 0.1.0` , `authenticated: false` , then `true` after adding `.env`	Confirms the MCP wiring works end to end, then that the token is loaded
Inspector → `search_repositories` `mcp server language:python` , limit 5	5 items, `total_count` 70,718	Real API call with the token; shows the trimming and the limit working
Inspector → `get_repository` `modelcontextprotocol/python-sdk`	13 fields incl. `license: "MIT"` , topics	Real detail view; checks the field mapping against real data
Inspector → `get_repository` `nobody-xyz/nothing`	“Not found. Check the owner and repository names…”	Proves real GitHub errors reach the model as readable messages
Inspector → `list_directory` / `get_file_contents` (step 3, in progress)	Expected: folders first; `truncated: true` on README with `max_chars` 2000; a folder error for `src`	Checks the context guards against a real repository
Inspector → `list_issues` limit 5, `list_pull_requests` state all (step 4, to do)	Expected: 5 issues with `has_more: true` ; PRs with `head` , `base` , `draft` ; `state: merged` rejected	Checks pagination and field mapping against real data
Inspector → `get_rate_limit` (step 5, to do)	Expected: `limit` 5000 with your token; `used` goes up after calling another tool	Confirms the token is used and the budget is read correctly
`git status` before committing	`.env` not listed	Proves the token can never be pushed to GitHub

Part 5. Commands you used

All in PowerShell, inside `C:\Users\prasa\projects\github-mcp-server`.

Command	What it does	Why
`powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"`	Installs **uv**	One fast tool for Python versions, virtual environments and dependencies
`uv --version`	Shows uv's version	Confirms the install worked
`uv sync`	Creates `.venv` and installs the exact versions in `uv.lock`	Reproducible environment on any machine
`uv run pytest`	Runs all tests inside the project environment	Proves everything works; run it after every change
`uv run mcp dev src/github_mcp/server.py`	Starts the server with the **MCP Inspector** in your browser	Lets you call tools by hand, like a host would
`Copy-Item .env.example .env` / `notepad .env`	Creates your local secrets file	Keeps the token out of code and git
`git init`	Starts a git repository in the folder	Version history
`git status`	Lists changed and new files	Check `.env` is **not** listed before committing
`git add .`	Stages all changes	Chooses what goes in the next commit
`git commit -m "..."`	Saves a snapshot with a message	One commit per step tells the project's story
`git config --global user.name/user.email`	Sets your identity for commits	Commits are credited to you on GitHub

How the Inspector works: `mcp dev` starts your server *and* a small web app. The web app acts as an MCP host: **Connect** sends `initialize`, **List Tools** sends `tools/list`, and **Run Tool** sends `tools/call`. You're doing by hand what Claude does automatically.

Part 6. Interview questions and model answers

MCP fundamentals

1. What problem does MCP solve? Without a standard, every AI app needs a custom integration for every service (N × M). MCP makes it N + M: wrap a service once as a server, and any MCP host can use it.

2. Host vs client vs server? The host is the AI app; a client is one connection inside the host; the server exposes capabilities and does the work.

3. Tool vs resource vs prompt? Tools are model-controlled actions; resources are application-controlled read-only data; prompts are user-selected templates.

4. What happens when a host connects to your server? `initialize` (versions, capabilities, instructions), then `tools/list` (names, descriptions, schemas), then `tools/call` whenever the model chooses a tool.

5. stdio vs Streamable HTTP? stdio is a local subprocess for one user with no network exposure. Streamable HTTP is a remote service for many clients and needs authentication (OAuth) and hardening.

6. Why can't you `print()` in a stdio server? stdout carries the JSON-RPC messages; extra output corrupts them. Log to stderr.

7. Which SDK version did you use, and what changed? MCP Python SDK v2. The high-level class is `MCPServer`, which was called `FastMCP` in v1.

Tool design

8. How does the model know what your tool does? From the tool name, description (docstring) and the JSON Schema generated from type hints and `Field` descriptions, all sent in `tools/list`.

9. What makes a good tool? A task-focused name, a description saying what it returns and when to use another tool, described parameters with examples, constraints in the schema, small typed outputs and actionable errors.

10. Why not expose every API endpoint as a tool? Models choose better from a small set of task-oriented tools. I design around what the model is trying to do, not the API's shape. For example, one contents endpoint became two tools: `list_directory` and `get_file_contents`.

11. What are tool annotations? Hints like `read_only_hint` and `open_world_hint` that hosts use for UX and safety, for example skipping confirmations for read-only tools. They're hints, not security.

12. How does the model see errors? As tool results with `isError: true` and a readable message. Messages say what went wrong and what to do (for example "Use list_directory"), so the model can correct itself.

Context and output

13. Why trim API responses? The context window is a budget. Fewer, relevant fields make calls cheaper and help the model decide better. Repositories go from ~100 fields to 13.

14. How do you stop a huge file flooding the context? Cut it off at `max_chars` with an explicit `truncated` flag, skip binary files, and reject files over 1 MB with a clear message.

15. Why tell the model a result was truncated? So it knows it has partial information and doesn't reason as if it saw everything.

16. How do you detect binary files? Null bytes or invalid UTF-8 mean binary. It's simple, and git uses the same heuristic.

Engineering

17. Why async? The work is I/O-bound. Async lets one process handle concurrent requests while waiting on GitHub.

18. Why one shared HTTP client? Connection pooling avoids repeated TCP/TLS handshakes; configuration lives in one place; cleanup happens in the lifespan.

19. What is the lifespan for? Creating shared resources at startup, closing them at shutdown (even after errors) and injecting them into tools through the context.

20. Why Pydantic models as return types? The SDK generates an output schema and returns `structuredContent`, so results are typed and predictable. Plain dicts didn't give structured output.

21. What's base64 and why does GitHub use it? An encoding of arbitrary bytes as safe ASCII text, so binary content can travel inside JSON.

Security

22. How do you handle the GitHub token? From an environment variable set by the host, or a git-ignored `.env` locally. It's wrapped in `SecretStr` so it never prints, and it's a fine-grained, read-only, expiring token (least privilege).

23. What if the token leaked? It can only read public repositories and it expires. I'd revoke it on GitHub immediately and create a new one.

Testing

24. How do you test code that calls an external API? I mock HTTP with `respx` and test success and failure paths (404, timeouts, binary and oversized files). Tests call tools through the SDK's in-process `Client`, just like a host, so they're fast and deterministic.

25. What's a pytest fixture? Reusable setup a test requests by parameter name. Ours, `github_api`, provides a fake GitHub that blocks real network calls.

Tools and choices

26. Why uv instead of pip? One fast tool handles Python versions, virtual environments and locked dependencies. `uv.lock` makes installs identical on every machine, and MCP hosts commonly launch servers with `uv run`.

27. Why httpx instead of requests? `requests` has no async support. httpx does, plus connection pooling, timeouts and base URLs, and `respx` can mock it cleanly.

28. Why not use a GitHub SDK like PyGithub? It's synchronous and hides the HTTP layer. I wanted control over headers, timeouts, error messages and, later, rate limits and retries.

29. Why mock GitHub in tests instead of calling it? Real calls are slow, need a token, hit rate limits, change over time and can't reliably produce failures like timeouts. Mocks make tests fast, deterministic and able to cover every error path. I verified against the real API separately with the Inspector.

30. What did you check manually, and why? I ran each tool in the MCP Inspector against real GitHub (search, details, a missing repo, truncation) to confirm that what the mocks assume matches reality, and checked `git status` to make sure `.env` was never committed.

Pagination and lists (step 4)

31. How do you handle pagination? I follow GitHub's `Link` header (`rel="next"`) until I have enough items, with a hard cap of 5 pages per call, and return `has_more` so the model knows the list is partial.

32. Offset vs cursor pagination? Offset (page numbers) is simple and allows jumping to a page, but can skip or duplicate items when data changes underneath. Cursors are stable and efficient but forward-only. Following `Link` headers works with both.

33. Why cap pages when the caller is an LLM? A model can ask for far more than it needs. A cap bounds latency, rate-limit usage and context size, whatever the model requests.

34. Why does your issues tool drop pull requests? GitHub treats every pull request as an issue, so `/issues` returns both. I filter on the `pull_request` key while paging and point the model to `list_pull_requests` .

35. What's the security risk in following pagination links? The client sends the token on every request, so a "next" URL to another host would leak it. I only follow URLs under `https://api.github.com/` (with the trailing slash, so look-alike hosts fail), and a test proves it.

36. Why use `Literal` for the `state` parameter? It becomes an enum in the JSON Schema: the model sees the allowed values, and invalid ones are rejected automatically.

Retries and rate limits (step 5)

37. Which errors do you retry, and which not? Timeouts, network errors and 5xx, because they're usually temporary. Not other 4xx, because the same request would fail the same way; the model has to change its input instead.

38. What is exponential backoff with jitter, and why the jitter? Each retry waits up to twice as long as the last (0.5 s, then 1 s). The actual wait is random within that ceiling, so many clients that failed together don't retry in lockstep and overload the server again.

39. What is idempotency and why does it matter for retries? Doing a request twice has the same effect as once. GETs are idempotent, so retrying is safe. A POST like `create_issue` isn't: if the response was lost, a retry could create a duplicate.

40. How do you handle GitHub's rate limits? I read `Retry-After` and the `X-RateLimit-*` headers. Waits of 10 seconds or less I wait out; longer ones I report with the reset time, because a tool call shouldn't hang. A `get_rate_limit` tool shows the remaining budget.

41. How do you test retries without slow tests? A fixture replaces the sleep function with a recorder, respx `side_effect` lists simulate fail-then-succeed, and the tests assert on the request count and the recorded delays.

42. Why write "retrying now will not help" in an error message? The error goes back to the model as text, so it works as a prompt. It should tell the model what to do next: stop and tell the user when the limit resets.

Part 7. Practise checklist

- [ ] Say the 30-second pitch out loud without notes.
- [ ] Draw the architecture diagram from memory.
- [ ] Explain the MCP handshake (`initialize` , `tools/list` , `tools/call`).
- [ ] Open `server.py` and explain `get_file_contents` line by line.
- [ ] Explain why `ctx: Context` doesn't appear in the tool's input schema.
- [ ] In the Inspector, trigger each error (missing repo, folder passed to `get_file_contents`) and explain what the model sees.
- [ ] Explain `get_paginated` line by line, including the same-host check.
- [ ] Explain the retry loop in `_get_response` : what is retried, what isn't, and why.
- [ ] Answer all 42 questions in your own words, then compare.